

NAVEGACIÓN Y GESTIÓN DEL ESTADO EN APLICACIONES ANDROID

Grupo 7

Introducción

El desarrollo de aplicaciones móviles modernas exige decisiones arquitectónicas fundamentadas que garanticen aplicaciones predecibles, mantenibles y confiables. Dos pilares centrales de esas decisiones son la navegación entre pantallas y la gestión del estado. En Android con Jetpack Compose, la navegación ha evolucionado hacia un modelo declarativo que centraliza la lógica de transición, gestiona automáticamente el historial y permite flujos coherentes. Por su parte, la gestión del estado aborda uno de los desafíos más subestimados del desarrollo móvil: a medida que una app crece, el estado se convierte en un sistema de datos interrelacionados que puede cambiar desde múltiples fuentes de forma asíncrona, generando inconsistencias si no se maneja correctamente.

Navigation Component en Compose

Navigation Component es una librería de Android Jetpack que estructura la navegación dentro de una aplicación. Se compone de tres elementos principales. El NavController controla la navegación y gestiona el historial (Back Stack) mediante funciones como navigate() y popBackStack(). El NavHost actúa como contenedor que muestra la pantalla actual, determinada por un grafo de navegación asociado al NavController. Los Destinations o destinos representan cada pantalla, definidos mediante la función composable() con una ruta única. El flujo de navegación sigue una estructura clara: se crea el NavController, se define el NavHost con una pantalla inicial, se registran las rutas y el usuario interactúa para que el NavController ejecute la navegación. Permite enviar argumentos entre pantallas, aunque se recomienda transmitir solo datos mínimos como identificadores. Sus ventajas incluyen reducción de código repetitivo, mantenimiento simplificado e integración completa con Jetpack Compose.

Flujo de Navegación entre Pantallas

El flujo de navegación no es simplemente cambiar de vista, sino un proceso que involucra cómo se apilan las pantallas, en qué orden se recorren y qué sucede con las anteriores. Su componente central es el back stack, una estructura LIFO (Last In, First Out) que registra cada pantalla visitada. Al navegar se agrega una pantalla al tope; al retroceder se elimina. Cuando un destino es sacado del back stack, su ciclo de vida se destruye y el ViewModel asociado es limpiado. Existen tres tipos de flujo: el lineal (avance pantalla por pantalla), el flujo con limpieza de stack usando popUpTo (para evitar que el usuario regrese a pantallas como el Login tras autenticarse) y el flujo circular con launchSingleTop=true (para prevenir duplicados en el stack). Las transiciones visuales también forman parte del flujo, con Android 15 introduciendo animaciones de retroceso predictivo ligadas al gesto del usuario. Un buen flujo debe ser predecible, mantener un historial claro, separar la lógica de navegación de las pantallas y controlar explícitamente el crecimiento del stack.

Manejo de Estados Complejos

El estado de una aplicación es la totalidad de la información que determina cómo se ve y comporta la interfaz en un momento dado. Un estado simple es una variable aislada con una sola fuente de cambio; un estado complejo es un sistema de variables interrelacionadas con múltiples fuentes de cambio (red, base de datos, sistema operativo), dependencias semánticas entre variables, naturaleza asíncrona y múltiples consumidores. La complejidad escala exponencialmente: con 10 variables booleanas existen 1.024 combinaciones posibles, la mayoría incoherentes. El estado se clasifica por alcance (local o compartido), duración (efímero o persistente) y naturaleza (de UI o de dominio). El problema central son los estados imposibles: combinaciones técnicamente representables, pero semánticamente inválidas, como tener isLoading=true y errorMessage no nulo al mismo tiempo. La solución es usar sealed classes o sealed interfaces, que reducen las combinaciones posibles exactamente a los casos válidos definidos, convirtiendo errores lógicos en errores de compilación. El patrón UI State Holder complementa esto encapsulando todo el estado de una pantalla en una data class única, garantizando actualizaciones atómicas mediante copy().

Single Source of Truth (SSOT)

El principio de Single Source of Truth establece que cada dato debe existir y gestionarse en un único lugar autorizado; cualquier otra parte del sistema debe consultarlo desde esa fuente original

en lugar de mantener su propia copia. Sin este principio, los datos se almacenan en múltiples lugares generando silos de datos con versiones contradictorias, dificultad para depurar errores y duplicación de esfuerzos al actualizar. En Android con Jetpack Compose, el SSOT se implementa en dos niveles: el ViewModel actúa como fuente única del estado de la UI, exponiéndolo mediante StateFlow o LiveData y siendo el único autorizado para modificarlo; el Repository actúa como fuente única de los datos de la aplicación, decidiendo si provienen de la red, la base de datos local o una caché, sin que ningún otro componente acceda directamente a esas fuentes. Sus beneficios incluyen consistencia garantizada, errores más fáciles de detectar, UI siempre sincronizada, código más mantenible y mejor testabilidad.

Manejo de Estados con ViewModel y StateFlow

El ViewModel es un componente de arquitectura que almacena y administra datos de la UI, sobreviviendo a cambios de configuración como la rotación de pantalla. Separa la lógica de presentación de los componentes visuales: los Composables solo muestran información, mientras el ViewModel la administra, ejecuta lógica de negocio y coordina operaciones con repositorios. StateFlow es una implementación de Flow de Kotlin Coroutines que siempre contiene un valor actual y notifica automáticamente a todos los observadores cuando cambia, permitiendo que los Composables se recompongan sin actualizaciones manuales. Juntos implementan el Flujo de Datos Unidireccional (UDF): el usuario genera un evento, el ViewModel lo procesa y actualiza el StateFlow, y los Composables reflejan el nuevo estado automáticamente. Este esquema (Usuario → Evento → ViewModel → StateFlow → Composable) garantiza un único punto de modificación del estado, reduce inconsistencias y materializa el principio SSOT en la práctica. Sus ventajas son persistencia ante cambios de configuración, actualización automática de la interfaz, separación de responsabilidades, mayor mantenibilidad y facilidad para realizar pruebas unitarias.

Conclusión

La navegación y la gestión del estado son decisiones de diseño que determinan directamente la calidad y confiabilidad de una aplicación Android. Navigation Component centraliza la lógica de navegación con un modelo declarativo, mientras el correcto manejo del back stack y los flujos garantiza experiencias de usuario coherentes. El principio SSOT, implementado mediante ViewModel y StateFlow, asegura que la interfaz siempre refleje una única versión consistente de la información. El modelado con sealed classes elimina los estados imposibles por diseño, transformando errores lógicos en errores de compilación. En conjunto, estos conceptos permiten construir aplicaciones robustas, escalables y preparadas para la complejidad real de producción, donde la asincronía y la escala son la norma.